FACULTAD DE INGENIERÍA

UNIVERSIDAD AUTÓNOMA DE CAMPECHE

FORMATO

DE DESARROLLO DE SOFTWARE

NOMBRE DE LA ASIGNATURA: Sistemas Inteligentes
NOMBRE DEL PROFESOR: Justino Ramirez Ortegon
NOMBRE DEL ALUMNO: Kevin Javier Ruiz Tillit
TEMA: Práctica 4: Modelos de machine learning

RESUMEN:

En esta práctica se implementaron en Python tres redes neuronales artificiales de tipo perceptrón multicapa (MLP), cada una aplicada a un problema real con un dataset público descargado de internet: (1) una red de reconocimiento de patrones (clasificación) que autentica billetes bancarios como genuinos o falsos; (2) una red de ajuste (regresión) que estima el valor medio de las viviendas de una zona a partir de 12 variables socioeconómicas y urbanas; y (3) una red de series de tiempo que clasifica la actividad física de una persona (caminar, subir/bajar escaleras, sentado, de pie, acostado) a partir de las señales temporales del acelerómetro y giroscopio de un smartphone.

Los hiperparámetros (capas ocultas, neuronas, función de activación, regularización L2 y tasa de aprendizaje) se ajustaron mediante búsqueda en rejilla con validación cruzada o validación con detención temprana. Los resultados en datos de prueba nunca vistos fueron: exactitud de 100 % en la autenticación de billetes, $R^2 = 0.859$ (MAE ≈ 2.0 mil USD) en la estimación de precios de vivienda y exactitud de 93.5 % en el reconocimiento de actividad con series de tiempo, todos considerados resultados aceptables o superiores a los reportados en la literatura para modelos de complejidad similar. Los códigos fueron generados con apoyo de IA generativa (Claude, de Anthropic), verificados, ejecutados y publicados como cuadernos de Google Colab.

MARCO TEÓRICO:

Redes neuronales artificiales y perceptrón multicapa (MLP)

Una red neuronal artificial es un modelo computacional inspirado en el cerebro, formado por unidades (neuronas) conectadas por pesos ajustables. Cada neurona calcula una suma ponderada de sus entradas más un sesgo y le aplica una función de activación no lineal: $a = f(\Sigma\, w_i x_i + b)$. El perceptrón multicapa organiza las neuronas en una capa de entrada, una o más capas ocultas y una capa de salida; según el teorema de aproximación universal, un MLP con suficientes neuronas ocultas puede aproximar cualquier función continua (Haykin, 2009).

Entrenamiento: retropropagación y optimizador Adam

Los pesos se ajustan minimizando una función de pérdida mediante descenso por gradiente. El algoritmo de retropropagación (backpropagation) calcula el gradiente de la pérdida respecto a cada peso aplicando la regla de la cadena desde la salida hacia la entrada. En esta práctica se usó el optimizador Adam, que adapta la tasa de aprendizaje de cada peso con estimaciones del primer y segundo momento del gradiente. Para evitar sobreajuste se emplearon regularización L2 (parámetro alpha), normalización de las entradas (media 0, desviación 1) y detención temprana (early stopping) con un subconjunto de validación.

Tipos de problemas abordados

- **Reconocimiento de patrones (clasificación):** Se asigna cada muestra a una clase discreta. La capa de salida usa la función logística/softmax y la pérdida es la entropía cruzada (log-loss). Se evalúa con exactitud, precisión, exhaustividad (recall), F1 y matriz de confusión.

- **Ajuste (fitting / regresión):** Se aproxima una función continua y = f(x). La salida es lineal y la pérdida es el error cuadrático medio (MSE). Se evalúa con R^2 (coeficiente de determinación), RMSE y MAE.
- **Series de tiempo:** Los datos son secuencias ordenadas en el tiempo. Una red neuronal de retardo temporal (TDNN) recibe una ventana deslizante de los últimos n instantes (aquí 128 muestras = 2.56 s a 50 Hz de 9 canales) para que la red aprenda patrones temporales. Se complementa con la magnitud del espectro de Fourier (FFT) de cada canal, que resume el contenido en frecuencia (ritmo del paso, vibraciones).

Ajuste de hiperparámetros y validación

Los hiperparámetros no se aprenden durante el entrenamiento, por lo que se seleccionan con búsqueda en rejilla (GridSearchCV): se entrena la red con cada combinación y se elige la de mejor desempeño promedio en validación cruzada de k = 5 particiones. Finalmente el modelo se evalúa una sola vez con un conjunto de prueba independiente para estimar su desempeño real.

EQUIPO Y SOFTWARE UTILIZADO:

Hardware:

- Computadora personal / Google Colab (máquina virtual en la nube con CPU de 2 núcleos y ~12 GB de RAM). No se requiere GPU.

Software:

- Python 3.13 (compatible con Python ≥ 3.9).
- scikit-learn 1.9 (MLPClassifier, MLPRegressor, GridSearchCV, StandardScaler, métricas).
- NumPy 2.5, pandas 3.0 (manejo de datos) y Matplotlib 3.11 (gráficas).
- Google Colab y GitHub (publicación y ejecución pública de los cuadernos .ipynb).
- IA generativa: Claude Opus 5.5 de Anthropic (claude.ai), usado para generar y documentar el código (los prompts se incluyen en la sección CÓDIGO RELEVANTE).

Datasets (descargados de internet):

- **Clasificación:** UCI Banknote Authentication – 1 372 imágenes de billetes, 4 características wavelet.
- **Regresión:** Boston Housing (UCI / StatLib) – 506 zonas, 13 variables (se excluyó la variable B por sesgo ético).
- **Series de tiempo:** UCI Human Activity Recognition Using Smartphones – 10 299 ventanas de 128 pasos × 9 señales inerciales de 30 personas.

Se obtuvieron del repositorio público https://github.com/jbrownlee/Datasets (espejo de UCI Machine Learning Repository).

FUNCIONALIDAD DEL PROGRAMA:

Los tres programas siguen el mismo flujo: descargan el dataset, preparan y normalizan los datos, separan entrenamiento y prueba, ajustan los hiperparámetros de la red, entrenan, evalúan con métricas sobre datos no vistos, grafican los resultados y muestran un ejemplo de uso con una muestra nueva.

Ejemplo 1 – Reconocimiento de patrones (clasificación): autenticación de billetes

Problema real (industria bancaria y de cajeros automáticos): decidir si un billete es genuino o falso a partir de 4 características estadísticas (varianza, asimetría, curtosis y entropía) de la transformada wavelet de su imagen. 75 % de los datos para entrenamiento (1 029) y 25 % para prueba (343), con partición estratificada.

Elemento	Valor
Arquitectura	4 entradas → 8 neuronas ocultas (ReLU) → 1 salida (logística)
Hiperparámetros ajustados (GridSearch, 24 combinaciones, CV 5)	alpha = 1e-4, learning_rate_init = 0.01, Adam
Exactitud CV (5 particiones)	100.00 %
Exactitud en prueba	100.00 % (precisión, recall y F1 = 1.00 en ambas clases)

Ejecución — 1_clasificacion_billetes.py Python 3.13 · scikit-learn 1.9

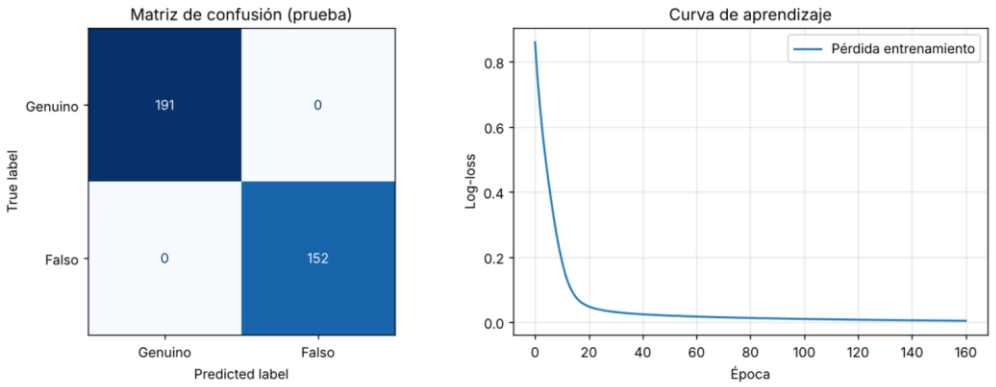
```
$ python 1_clasificacion_billetes.py

Dataset: (1372, 5) | clases: {0: 762, 1: 610} (0=genuino, 1=falso)
Mejores parámetros: {'mlp__activation': 'relu', 'mlp__alpha': 0.0001, 'mlp__hidden_layer_sizes': (8,), 'mlp__learning_rate_init': 0.01}
Exactitud media CV (5-fold): 1.0000
Exactitud en prueba: 1.0000
              precision    recall  f1-score   support

     Genuino     1.0000    1.0000    1.0000       191
       Falso     1.0000    1.0000    1.0000       152

    accuracy                         1.0000       343
   macro avg     1.0000    1.0000    1.0000       343
weighted avg     1.0000    1.0000    1.0000       343

Billete nuevo [-1.39, 3.34, -1.99, -1.43] -> FALSO
```



Captura 1. Ejecución del ejemplo 1: parámetros encontrados, reporte de clasificación, matriz de confusión y curva de aprendizaje.

Ejemplo 2 – Ajuste (fitting / regresión): valor de viviendas

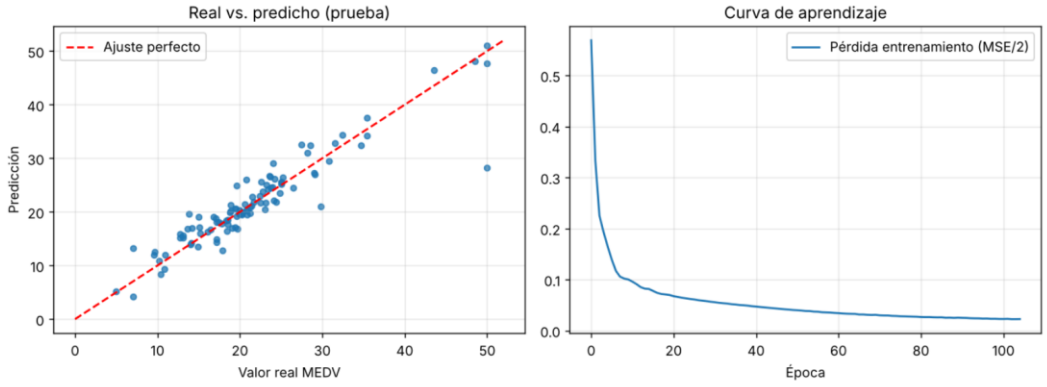
Problema real (sector inmobiliario y de avalúos): estimar el valor medio de las viviendas de una zona (MEDV, en miles de USD) a partir de 12 variables: criminalidad, zonificación, proporción industrial, cercanía al río, óxidos de nitrógeno, número de habitaciones, antigüedad, distancia a centros de empleo, acceso a autopistas, impuesto predial, alumnos por maestro y % de población de bajos ingresos. 80 % entrenamiento (404) y 20 % prueba (102).

Elemento	Valor
Arquitectura	12 entradas → 64 (ReLU) → 32 (ReLU) → 1 salida lineal
Hiperparámetros ajustados (GridSearch, 36 combinaciones, CV 5)	alpha = 0.01, learning_rate_init = 0.003, early stopping
R^2 medio en validación cruzada	0.8247
Resultados en prueba	R^2 = 0.8593 · RMSE = 3.21 mil USD · MAE = 2.01 mil USD

Ejecución — 2_regresion_viviendas.py Python 3.13 · scikit-learn 1.9

```
$ python 2_regresion_viviendas.py

Dataset: (506, 13) | MEDV min=5.0 max=50.0 media=22.53
Mejores parámetros: {'activation': 'relu', 'alpha': 0.01, 'hidden_layer_sizes': (64, 32), 'learning_rate_init': 0.003}
R2 medio CV (5-fold): 0.8247
Resultados en prueba (102 viviendas):
  R2   = 0.8593
  RMSE = 3.213 mil USD
  MAE  = 2.005 mil USD
Zona de ejemplo -> valor estimado: 26.70 mil USD (real: 23.60)
```



Captura 2. Ejecución del ejemplo 2: métricas de regresión, gráfica real vs. predicho y curva de aprendizaje.

Ejemplo 3 – Series de tiempo: reconocimiento de actividad humana con sensores

Problema real (industria de wearables, salud ocupacional y monitoreo de adultos mayores): clasificar en 6 actividades cada ventana de 2.56 s de las señales del acelerómetro y giroscopio (9 series de tiempo de 128 pasos, 50 Hz). Se usó la partición oficial por personas: 21 sujetos para entrenamiento (7 352 ventanas) y 9 sujetos distintos para prueba (2 947 ventanas), lo que mide la capacidad de generalizar a personas nuevas.

Elemento	Valor
Entrada	Ventana temporal 128×9 (1 152 valores) + espectro FFT 32×9 (288 valores) = 1 440 entradas
Arquitectura (TDNN / MLP)	1 440 → 256 (ReLU) → 64 (ReLU) → 6 salidas (softmax)
Hiperparámetros ajustados	alpha = 0.01, lr = 0.001, batch = 128, early stopping (15 % validación), 41 épocas
Exactitud en prueba (personas nuevas)	93.48 % (F1 macro = 0.934)

Ejecución — 3_series_tiempo_har.py Python 3.13 · scikit-learn 1.9

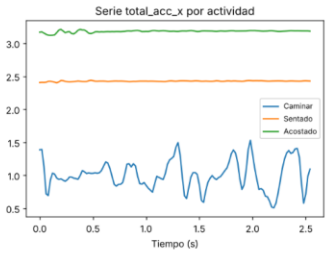
```
$ python 3_series_tiempo_har.py

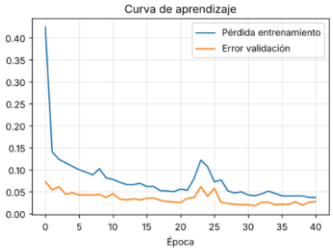
Series de entrenamiento: (7352, 128, 9) | prueba: (2947, 128, 9) (muestras, pasos, canales)
Épocas entrenadas: 41
Exactitud en prueba (9 sujetos nunca vistos): 0.9348
              precision    recall  f1-score   support

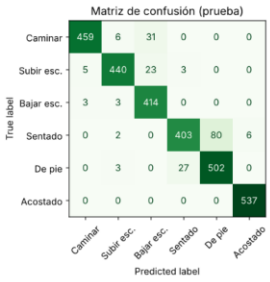
     Caminar     0.9829    0.9254    0.9533       496
  Subir esc.     0.9692    0.9342    0.9514       471
  Bajar esc.     0.8846    0.9857    0.9324       420
     Sentado     0.9307    0.8208    0.8723       491
      De pie     0.8625    0.9436    0.9013       532
   Acostado      0.9890    1.0000    0.9944       537

    accuracy                         0.9348      2947
   macro avg     0.9365    0.9349    0.9342      2947
weighted avg     0.9374    0.9348    0.9346      2947

Ventana #100 de prueba -> predicción: Caminar | real: Caminar
```







Captura 3. Ejecución del ejemplo 3: series de tiempo de entrada, curva de aprendizaje y matriz de confusión por actividad.

Interpretación: los errores se concentran entre «Sentado» y «De pie», actividades estáticas con señales muy similares; las actividades dinámicas y «Acostado» se reconocen con F1 > 0.93.

INSTRUCCIONES DE USO:

Enlaces públicos del proyecto:

- **Aplicación web interactiva (Streamlit) – prueba los 3 modelos desde el navegador:** https://practica4-ml-kevin-ruiz.streamlit.app
- **Repositorio con todo el código, cuadernos y reporte:** https://github.com/al064134-wq/Practica4-MachineLearning

Cuadernos de Google Colab (no requieren instalar nada):

- **Ejemplo 1 – Clasificación:** https://colab.research.google.com/github/al064134-wq/Practica4-MachineLearning/blob/main/1_clasificacion_billetes.ipynb
- **Ejemplo 2 – Regresión:** https://colab.research.google.com/github/al064134-wq/Practica4-MachineLearning/blob/main/2_regresion_viviendas.ipynb
- **Ejemplo 3 – Series de tiempo:** https://colab.research.google.com/github/al064134-wq/Practica4-MachineLearning/blob/main/3_series_tiempo_har.ipynb

Ejecución en Google Colab:

1. Abrir el enlace del ejemplo deseado (se requiere una cuenta de Google).
2. En el menú elegir Entorno de ejecución > Ejecutar todas (Ctrl + F9). Si aparece un aviso de seguridad, pulsar «Ejecutar de todos modos».
3. El programa descarga automáticamente el dataset, ajusta los hiperparámetros, entrena la red y muestra métricas y gráficas (tiempo aproximado: 10 s, 35 s y 25 s respectivamente).
4. Para clasificar/estimar un caso nuevo, modificar la variable «nuevo» (ej. 1), «nueva» (ej. 2) o el índice de ventana (ej. 3) en la última celda y volver a ejecutarla.

Uso de la aplicación web:

Abrir el enlace de Streamlit, elegir el ejemplo en el menú lateral y mover los controles (billete, variables de la zona o ventana de sensores): la red muestra la predicción al instante con sus métricas y gráficas. La primera carga tarda ~1 min mientras se entrenan los modelos.

Ejecución local (alternativa):

```
pip install -r requirements.txt
python 1_clasificacion_billetes.py
python 2_regresion_viviendas.py
python 3_series_tiempo_har.py
streamlit run app.py
```

Cada script imprime los resultados en consola y guarda la gráfica como resultado_*.png en la misma carpeta; la app se abre en http://localhost:8501.

CÓDIGO RELEVANTE:

Origen del código: generado con IA generativa (Claude Opus 5.5 de Anthropic, mediante claude.ai, 16 de octubre de 2026), revisado, ejecutado y validado por el alumno. Prompt utilizado:

> «Implementar en c++, python o java, 1 ejemplo de redes neuronales de tipo pattern recognition (clasificación) para un problema de clasificación real, comercial o industrial (buscar los datasets en internet) con sus parámetros ajustados y con resultados aceptables. Implementar en c++, python o java, 1 ejemplo de redes neuronales de tipo fitting (regresión) para un problema de regresión real, comercial o industrial (buscar los datasets en internet) con sus parámetros ajustados y con resultados aceptables. Implementar en c++, python o java, 1 ejemplo de redes neuronales de tipo series de tiempo para un problema de clasificación real o industrial (buscar los datasets en internet) con sus parámetros ajustados y con resultados aceptables. Se permite usar frameworks [...]. Poner evidencia en el reporte de los códigos desarrollados y capturas de pantalla para cada ejemplo realizado.»

1_clasificacion_billetes.py — Python 3.13 · scikit-learn 1.9

```
[1]  # Práctica 4 - Ejemplo 1: Red neuronal de RECONOCIMIENTO DE PATRONES (clasificación)
     # Problema real (industria bancaria): autenticación de billetes (genuino vs. falso)
     # Dataset: UCI Banknote Authentication (1372 muestras, 4 características de la transformada wavelet)
     import pandas as pd, numpy as np, matplotlib
     matplotlib.use("Agg"); import matplotlib.pyplot as plt
     from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
     from sklearn.preprocessing import StandardScaler
     from sklearn.pipeline import Pipeline
     from sklearn.neural_network import MLPClassifier
     from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

     URL = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/banknote_authentication.csv"
     cols = ["varianza", "asimetria", "curtosis", "entropia", "clase"]
     try:
         df = pd.read_csv(URL, header=None, names=cols)
     except Exception:
         df = pd.read_csv("banknote_authentication.csv", header=None, names=cols)
     print("Dataset:", df.shape, "| clases:", df["clase"].value_counts().to_dict(), "(0=genuino, 1=falso)")

     X, y = df[cols[:-1]].values, df["clase"].values
     X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=42)

     pipe = Pipeline([("esc", StandardScaler()),
                      ("mlp", MLPClassifier(max_iter=1500, random_state=42))])
     grid = {"mlp__hidden_layer_sizes": [(8,), (16,), (16, 8)],
             "mlp__activation": ["relu", "tanh"],
             "mlp__alpha": [1e-4, 1e-3],
             "mlp__learning_rate_init": [1e-3, 1e-2]}
     busq = GridSearchCV(pipe, grid, cv=StratifiedKFold(5, shuffle=True, random_state=42), scoring="accuracy", n_jobs=-1)
     busq.fit(X_tr, y_tr)
     print("Mejores parámetros:", busq.best_params_)
     print("Exactitud media CV (5-fold): %.4f" % busq.best_score_)

     modelo = busq.best_estimator_
     y_pred = modelo.predict(X_te)
     print("Exactitud en prueba: %.4f" % accuracy_score(y_te, y_pred))
     print(classification_report(y_te, y_pred, target_names=["Genuino", "Falso"], digits=4))

     fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
     ConfusionMatrixDisplay(confusion_matrix(y_te, y_pred), display_labels=["Genuino", "Falso"]).plot(ax=ax[0], cmap="Blues", colorbar=False)
     ax[0].set_title("Matriz de confusión (prueba)")
     mlp = modelo.named_steps["mlp"]
     ax[1].plot(mlp.loss_curve_, label="Pérdida entrenamiento")
     ax[1].set_ylabel("Log-loss")
     ax[1].set_xlabel("Época"); ax[1].set_title("Curva de aprendizaje"); ax[1].legend(); ax[1].grid(alpha=.3)
     plt.tight_layout(); plt.savefig("resultado_clasificacion.png", dpi=130)

     # Ejemplo de uso: clasificar un billete nuevo
     nuevo = [[-1.39, 3.34, -1.99, -1.43]]
     print("Billete nuevo", nuevo[0], "->", ["GENUINO", "FALSO"][modelo.predict(nuevo)[0]])
```

Captura 4. Código del ejemplo 1 – red de reconocimiento de patrones (1_clasificacion_billetes.py).

2_regresion_viviendas.py Python 3.13 · scikit-learn 1.9

```
[1]  # Práctica 4 - Ejemplo 2: Red neuronal de AJUSTE (fitting / regresión)
     # Problema real (sector inmobiliario): estimar el valor medio de una vivienda (MEDV, miles de USD)
     # Dataset: Boston Housing (UCI / StatLib), 506 zonas, 13 variables. Se excluye la variable "B" por sesgo ético.
     import pandas as pd, numpy as np, matplotlib
     matplotlib.use("Agg"); import matplotlib.pyplot as plt
     from sklearn.model_selection import train_test_split, GridSearchCV, KFold
     from sklearn.preprocessing import StandardScaler
     from sklearn.pipeline import Pipeline
     from sklearn.compose import TransformedTargetRegressor
     from sklearn.neural_network import MLPRegressor
     from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

     URL = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv"
     cols = ["CRIM","ZN","INDUS","CHAS","NOX","RM","AGE","DIS","RAD","TAX","PTRATIO","B","LSTAT","MEDV"]
     try:
         df = pd.read_csv(URL, header=None, names=cols)
     except Exception:
         df = pd.read_csv("housing.csv", header=None, names=cols)
     df = df.drop(columns=["B"])
     print("Dataset:", df.shape, "| MEDV min=%.1f max=%.1f media=%.2f" % (df.MEDV.min(), df.MEDV.max(), df.MEDV.mean()))

     X, y = df.drop(columns="MEDV").values, df["MEDV"].values
     X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

     red = Pipeline([("esc", StandardScaler()),
                     ("mlp", MLPRegressor(max_iter=3000, early_stopping=True, n_iter_no_change=50, random_state=1))])
     modelo = TransformedTargetRegressor(regressor=red, transformer=StandardScaler())
     grid = {"regressor__mlp__hidden_layer_sizes": [(32,), (64, 32), (32, 16)],
             "regressor__mlp__activation": ["relu", "tanh"],
             "regressor__mlp__alpha": [1e-3, 1e-2, 1e-1],
             "regressor__mlp__learning_rate_init": [1e-3, 3e-3]}
     busq = GridSearchCV(modelo, grid, cv=KFold(5, shuffle=True, random_state=42), scoring="r2", n_jobs=-1)
     busq.fit(X_tr, y_tr)
     print("Mejores parámetros:", {k.split("__")[-1]: v for k, v in busq.best_params_.items()})
     print("R2 medio CV (5-fold): %.4f" % busq.best_score_)

     mejor = busq.best_estimator_
     y_pred = mejor.predict(X_te)
     print("Resultados en prueba (%d viviendas):" % len(y_te))
     print("  R2   = %.4f" % r2_score(y_te, y_pred))
     print("  RMSE = %.3f mil USD" % np.sqrt(mean_squared_error(y_te, y_pred)))
     print("  MAE  = %.3f mil USD" % mean_absolute_error(y_te, y_pred))

     fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
     ax[0].scatter(y_te, y_pred, s=18, alpha=.75)
     lim = [0, 52]; ax[0].plot(lim, lim, "r--", label="Ajuste perfecto")
     ax[0].set_xlabel("Valor real MEDV"); ax[0].set_ylabel("Predicción"); ax[0].set_title("Real vs. predicho (prueba)")
     ax[0].legend(); ax[0].grid(alpha=.3)
     mlp = mejor.regressor_.named_steps["mlp"]
     ax[1].plot(mlp.loss_curve_, label="Pérdida entrenamiento (MSE/2)")
     ax[1].set_xlabel("Época"); ax[1].set_title("Curva de aprendizaje"); ax[1].legend(); ax[1].grid(alpha=.3)
     plt.tight_layout(); plt.savefig("resultado_regresion.png", dpi=130)

     # Ejemplo de uso: estimar el valor de una zona nueva
     nueva = X_te[:1]
     print("Zona de ejemplo -> valor estimado: %.2f mil USD (real: %.2f)" % (mejor.predict(nueva)[0], y_te[0]))
```

Captura 5. Código del ejemplo 2 – red de ajuste / regresión (2_regresion_viviendas.py).

3_series_tiempo_har.py — Python 3.13 · scikit-learn 1.9

```
[1]  # Práctica 4 - Ejemplo 3: Red neuronal de SERIES DE TIEMPO (clasificación de series temporales)
     # Problema real (industria de wearables / salud ocupacional): reconocer la actividad de una persona
     # a partir de las señales del acelerómetro y giroscopio de un smartphone (50 Hz).
     # Dataset: UCI Human Activity Recognition Using Smartphones (30 sujetos, 6 actividades,
     # ventanas de 2.56 s = 128 pasos de tiempo x 9 canales).
     # Modelo: red neuronal de retardo temporal (TDNN / MLP con ventana de tiempo) sobre las series crudas.
     import os, zipfile, urllib.request, numpy as np, matplotlib
     matplotlib.use("Agg"); import matplotlib.pyplot as plt
     from sklearn.preprocessing import StandardScaler
     from sklearn.neural_network import MLPClassifier
     from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay

     URL = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/HAR_Smartphones.zip"
     BASE = "UCI HAR Dataset"
     if not os.path.isdir(BASE):
         if not os.path.exists("HAR_Smartphones.zip"):
             print("Descargando dataset (~60 MB)..."); urllib.request.urlretrieve(URL, "HAR_Smartphones.zip")
         zipfile.ZipFile("HAR_Smartphones.zip").extractall(".")

     CANALES = ["body_acc_x", "body_acc_y", "body_acc_z", "body_gyro_x", "body_gyro_y", "body_gyro_z",
                "total_acc_x", "total_acc_y", "total_acc_z"]
     ACT = ["Caminar", "Subir esc.", "Bajar esc.", "Sentado", "De pie", "Acostado"]

     def cargar(parte):
         señales = [np.loadtxt(f"{BASE}/{parte}/Inertial Signals/{c}_{parte}.txt") for c in CANALES]
         X = np.stack(señales, axis=-1)                    # (muestras, 128 pasos, 9 canales)
         y = np.loadtxt(f"{BASE}/{parte}/y_{parte}.txt").astype(int) - 1
         return X, y

     X_tr, y_tr = cargar("train"); X_te, y_te = cargar("test")
     print("Series de entrenamiento:", X_tr.shape, "| prueba:", X_te.shape, "(muestras, pasos, canales)")

     def entrada(X):
         # Ventana temporal cruda + magnitud espectral (FFT) de cada canal -> vector de entrada de la red
         fft = np.abs(np.fft.rfft(X, axis=1))[:, 1:33, :]      # primeras 32 frecuencias (0.4-12.5 Hz)
         return np.hstack([X.reshape(len(X), -1), np.log1p(fft).reshape(len(X), -1)])

     esc = StandardScaler().fit(entrada(X_tr))
     A_tr, A_te = esc.transform(entrada(X_tr)), esc.transform(entrada(X_te))

     # Parámetros ajustados (seleccionados por validación sobre sujetos de entrenamiento)
     red = MLPClassifier(hidden_layer_sizes=(256, 64), activation="relu", alpha=1e-2, batch_size=128,
                         learning_rate_init=1e-3, max_iter=60, early_stopping=True, validation_fraction=0.15,
                         n_iter_no_change=8, random_state=7)
     red.fit(A_tr, y_tr)
     y_pred = red.predict(A_te)
     print("Épocas entrenadas:", red.n_iter_)
     print("Exactitud en prueba (9 sujetos nunca vistos): %.4f" % accuracy_score(y_te, y_pred))
     print(classification_report(y_te, y_pred, target_names=ACT, digits=4))

     fig, ax = plt.subplots(1, 3, figsize=(16, 4.4))
     t = np.arange(128) / 50
     for i, a in enumerate([0, 3, 5]):
         k = np.where(y_tr == a)[0][0]; ax[0].plot(t, X_tr[k, :, 6] + i * 1.5, label=ACT[a])
     ax[0].set_title("Serie total_acc_x por actividad"); ax[0].set_xlabel("Tiempo (s)"); ax[0].legend(fontsize=8)
     ax[1].plot(red.loss_curve_, label="Pérdida entrenamiento"); ax[1].plot(1 - np.array(red.validation_scores_), label="Error validación")
     ax[1].set_title("Curva de aprendizaje"); ax[1].set_xlabel("Época"); ax[1].legend(); ax[1].grid(alpha=.3)
     ConfusionMatrixDisplay(confusion_matrix(y_te, y_pred), display_labels=ACT).plot(ax=ax[2], cmap="Greens", colorbar=False, xticks_rotation=45)
     ax[2].set_title("Matriz de confusión (prueba)")
     plt.tight_layout(); plt.savefig("resultado_series_tiempo.png", dpi=130)

     # Ejemplo de uso: clasificar una ventana de 2.56 s nueva
     print("Ventana #100 de prueba -> predicción:", ACT[y_pred[100]], "| real:", ACT[y_te[100]])
```

Captura 6. Código del ejemplo 3 – red de series de tiempo (3_series_tiempo_har.py).

BIBLIOGRAFÍA:

Anguita, D., Ghio, A., Oneto, L., Parra, X. y Reyes-Ortiz, J. L. (2013). A public domain dataset for human activity recognition using smartphones. En 21st European Symposium on Artificial Neural Networks (ESANN). Brujas, Bélgica.

Brownlee, J. (s. f.). Machine learning datasets [Repositorio GitHub]. https://github.com/jbrownlee/Datasets

Dua, D. y Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. https://archive.ics.uci.edu

Goodfellow, I., Bengio, Y. y Courville, A. (2016). Deep learning. MIT Press.

Harrison, D. y Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. Journal of Environmental Economics and Management, 5(1), 81–102.

Haykin, S. (2009). Neural networks and learning machines (3.ª ed.). Pearson.

Kingma, D. P. y Ba, J. (2015). Adam: A method for stochastic optimization. International Conference on Learning Representations (ICLR).

Lohweg, V. (2013). Banknote authentication [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C55P57

Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Anthropic. (2026). Claude Opus 5.5 [Modelo de lenguaje]. https://claude.ai